

CompareWise AI

Project Description:

1. Project Overview :

CompareWise AI is an enterprise-grade, offline-first procurement intelligence platform designed to automate the extraction, analysis, and comparison of vendor proposals. By leveraging local Large Language Models (LLMs), the platform allows organizations to securely analyze complex PDF, DOCX, and XLSX documents without exposing sensitive data to cloud APIs.

2. Problem Statement :

Procurement managers and enterprise teams spend countless hours manually reading, comparing, and summarizing lengthy vendor proposals. Traditional methods are prone to human error, incredibly time-consuming, and struggle to efficiently identify hidden risk clauses or normalize pricing across disparate proposal formats. Furthermore, using cloud-based AI tools for this poses a severe data privacy risk.

3. Objectives :

Key Project Capabilities: Transforming Vendor Evaluation

This project is engineered to fundamentally change how organizations evaluate and select vendors, moving from labor-intensive, error-prone manual analysis to an efficient, AI-driven process. The core capabilities are designed to deliver speed, accuracy, and absolute security to the decision-making pipeline.

1. Accelerated and Informed Decision-Making:

- **Original Focus:** Facilitate rapid comparisons with clear, actionable, side-by-side executive views of vendor proposals.
- **Elaboration:** The platform provides a dynamic, single-pane-of-glass executive dashboard that instantly synthesizes data from multiple, diverse vendor submissions. This removes the need for manual data collation and spreadsheet management. By presenting key metrics, comparative scores, and risk profiles in an easy-to-digest, side-by-side format, the system enables executive stakeholders to achieve consensus and make highly informed decisions in a fraction of the time traditionally required. This acceleration drastically shortens the procurement cycle, allowing organizations to realize project value sooner.

2. Advanced Automated Insights Extraction:

- **Original Focus:** Automatically extract essential project metrics—including cost, timeline, service level agreements (SLAs), deliverables, and risks—from complex, unstructured vendor documents.
- **Elaboration:** Leveraging state-of-the-art Natural Language Processing (NLP) and Machine Learning (ML) models, the system autonomously parses voluminous, complex, and unstructured documents—such as RFPs, SOWs, and legal agreements. It goes beyond simple keyword extraction by understanding the semantic context to identify and quantify critical data points, including:

- **Financial Metrics:** Total Cost of Ownership (TCO), payment milestones, and pricing structures.
- **Project Governance:** Detailed timelines, key milestones, and required resources.
- **Contractual Obligations:** Specific Service Level Agreements (SLAs), performance metrics, and penalty clauses.
- **Risk Identification:** Highlighting non-standard contractual terms, potential compliance issues, and ambiguous language that could pose a future risk to the organization. This automated process ensures 100% data capture and eliminates human error during the transcription and analysis phase.

3. **Guaranteed Data Privacy and Confidentiality:**

- **Original Focus:** Ensure complete data confidentiality by performing all artificial intelligence (AI) processing exclusively on the user's local hardware.
- **Elaboration:** In an era of heightened data security concerns, this capability is paramount. The system is architected on an "on-premise" or "local hardware" processing model. This critical design choice means that sensitive vendor proposals, confidential pricing, and proprietary organizational data **never leave the user's secured internal network or device** for AI analysis. All computational intensive tasks, including model training, inference, and data extraction, are performed locally. This architecture guarantees compliance with stringent internal

security policies and external regulatory requirements (e.g., GDPR, CCPA), providing stakeholders with absolute confidence in the confidentiality of their strategic procurement information.

4. Core Technologies Used

Core Technologies & StackBackend

- **Python:** Serves as the primary backend language. It is responsible for core functionalities like file processing, handling asynchronous operations, and routing API requests.
- **FastAPI:** A modern, high-performance, and fast web framework used to establish the backend API endpoints and manage the serving of static frontend files.
- **Ollama (Llama 3.2):** This local AI framework runs the Llama 3.2 model entirely on-device, replacing the need for external APIs (like Groq or OpenAI). It is used for secure semantic text extraction and executive summarization, ensuring no data leakage occurs.

Frontend

- **Vanilla Web Stack (HTML, CSS, JS):** A lightweight, dependency-free frontend implementation. Key features include a premium glassmorphism design, dynamic DOM manipulation, and a simulated progress tracking experience.

5. Key Features

The core capabilities of the system are as follows:

Data Ingestion & Parsing

- **Multi-Format Document Parsing:** Features a robust ingestion engine designed for accurately extracting raw text from diverse file types, including **.pdf**, **.docx**, and **.xlsx** documents.
- **Smart File Caching:** Improves efficiency by using MD5 hashing to remember and quickly return results in milliseconds for proposals that have been processed previously.

AI-Powered Analysis

- **AI Metric Extraction Engine:** Employs local Large Language Models (LLMs) for semantic understanding, extracting and normalizing critical values such as Total Cost, Timeline, Service Level Agreements (SLAs), and Key Deliverables, irrespective of the vendor's document formatting.
- **Risk Clause Detection:** Proactively scans documents to identify and flag potential liabilities, hidden fees, or unfavorable termination clauses, aiding in risk mitigation.

Output & Recommendation

- **AI Executive Summaries:** Generates high-level summaries that evaluate the pros and cons of each vendor, concluding with a definitive, AI-driven recommendation.
- **Curated Vendor Recommendations:** Provides a dedicated User Interface (UI) hub containing pre-loaded, best-in-class proposals that users can add to the comparison engine with a single click.

7. Real-World Applications

Application Scenarios / Use Cases
Scenario 1: Enterprise Procurement & RFP Analysis

Procurement teams drop RFP responses to instantly generate a normalized comparison matrix, cutting vendor selection time from weeks to minutes.
Scenario 2: IT Software Vendor Selection

IT managers comparing complex software SLAs, licensing, and implementation timelines easily spot hidden out-of-scope costs.
Scenario 3: Event Management & Logistics

Event planners compare quotes from Stage, Decor, Tent, Food, and Photography vendors simultaneously for the most cost-effective budget.
Scenario 4: Legal & Risk Mitigation

Legal teams use the risk detection engine as a first-pass filter to quickly highlight unfavorable clauses before deep manual review.

TECHNICAL ARCHITECTURE :

System Architecture and Layered Breakdown

The application is structured into distinct, logical layers, designed for clarity, maintainability, and optimal performance, especially considering the constraints of local execution and data privacy.
Frontend Layer: User Interaction and State Management

The frontend is a **Single Page Application (SPA)**, intentionally built with **Vanilla JavaScript** to minimize dependencies and ensure rapid load times. It serves as the primary interface for the user, managing all client-side logic.

- **State Management:** It maintains the entire application state

client-side, ensuring a fluid and responsive user experience without constant server round-trips.

- **File Handling:** It incorporates robust handling for **drag-and-drop file inputs**, providing users with an intuitive way to upload comparison documents.
- **Visual Feedback:** To manage user expectations during potentially long processing times, the frontend visually **simulates AI processing progress**. This includes a progress bar or status indicator that updates as the backend performs LLM operations.
- **Dynamic Rendering:** It is responsible for **dynamically rendering the comparison tables and vendor cards** once the AI analysis is complete, presenting the structured output in an easily digestible format.

Backend Layer: API and Asynchronous Processing

The backend is built on **FastAPI**, chosen for its high performance and native support for asynchronous operations (**async/await**). It serves as the bridge between the client and the core AI logic.

- **Request Handling:** The server efficiently handles **incoming multipart/form-data uploads** from the client, processing the raw file inputs.
- **Asynchronous Offloading:** Crucially, to maintain a responsive event loop (a core feature of FastAPI/Uvicorn), it uses **asyncio.to_thread**. This function is specifically employed to offload two types of blocking operations to a separate thread pool:

- **Blocking File I/O:** Reading and processing large uploaded files.
- **Heavy LLM Processing:** The synchronous calls made to the Ollama REST API for AI analysis.
- *Goal:* This architecture prevents these potentially time-consuming tasks from blocking the main thread, ensuring the server remains highly available and fast for other requests.

AI / Logic Layer: Local LLM Execution and Data Consistency

The core intelligence of the application resides in this layer, which leverages a local LLM environment to perform the required analysis.

- **Local Execution Environment:** The LLM is run using **Ollama**, which is configured to operate locally on the default port, **11434**. This choice eliminates reliance on cloud-based LLM APIs, ensuring faster turnaround and greater privacy.
- **Communication Protocol:** The backend communicates with the local Ollama instance exclusively via its **REST API**.
- **Constraint and Consistency:** A critical design feature is the use of **highly constrained prompts**. The prompts are engineered to force the LLM to focus narrowly on the comparison task. Furthermore, the system **forces JSON schema outputs**, ensuring that the unstructured text output of the LLM is reliably converted into a predictable, structured data format required for rendering the comparison tables. This mechanism is key to maintaining **data consistency** and reliability.

Data / Storage Layer: Privacy-Centric Caching

This layer is characterized by its strict adherence to user privacy and efficiency through in-memory storage.

- **Privacy Mandate:** To ensure **maximum privacy**, the system explicitly avoids using any **external database** (SQL, NoSQL, etc.). No user data or session information is persisted to disk in a persistent database.
- **Session Persistence Mechanism:** Instead of a database, the system utilizes an **in-memory dictionary-based caching system**.
 - **Key:** Each uploaded file is hashed using the **MD5 algorithm**, and this hash serves as the unique key for the cache entry.
 - **Value:** The corresponding LLM analysis results are stored as the value.
 - **Purpose:** This system provides temporary **session persistence** and ensures that if the same file is uploaded again during the active session, the heavy LLM processing is skipped, and the results are returned instantly from the cache, improving efficiency. Data is volatile and lost upon server restart.

Deployment Environment: Local First

The application is explicitly designed for a **local-first deployment model**.

- **Target Platform:** The primary target environment is **Windows machines**.
- **Execution Runtime:** The application is run via **Uvicorn**, which is the ASGI server that hosts the FastAPI application. This setup allows for

easy local installation and execution, bypassing the need for complex cloud infrastructure.

Pre-requisites:

System Requirements and Project Prerequisites

This project outlines a sophisticated local Large Language Model (LLM) application, requiring specific software, libraries, and hardware for optimal performance and functionality. Software Requirements

To successfully run and develop this application, the following foundational software must be installed and operational:

- **Python (Version 3.9+):** The core programming language for the backend. We specifically require Python 3.9 or a later release to leverage modern language features, performance improvements, and compatibility with the listed dependencies.
- **Ollama Application:** This is the crucial component for running local LLMs. Ollama must be installed on the host machine and actively running as a local service. It handles model management, execution, and providing an accessible API for the Python client.
- **Modern Web Browser:** For accessing and interacting with the application's front-end interface. Supported browsers include recent stable versions of Google Chrome, Microsoft Edge, and Mozilla Firefox. Compatibility with other browsers is not guaranteed.

Libraries, Frameworks, and Dependencies

The application relies on a collection of Python libraries and frameworks, categorized by their function: **Core Backend & API:**

- **fastapi**: The high-performance, modern framework used to build the asynchronous API endpoints.
- **uvicorn**: An ASGI server that runs the FastAPI application, necessary for handling concurrent connections and I/O-bound tasks efficiently.
- **python-multipart**: A dependency required by FastAPI to handle file uploads (e.g., documents for processing) via form data.

Local LLM Interaction:

- **ollama** (Python client): The official Python library used to communicate with the locally running Ollama server, facilitating model prompting and response generation.

Document Processing & Data Handling:

- **PyPDF2**: Used for reading, manipulating, and extracting text content from PDF documents.
- **python-docx**: Enables the application to read and process content from Microsoft Word (**.docx**) files.
- **openpyxl**: A library dedicated to reading and writing Excel files in the modern **.xlsx** format, primarily for structured data extraction.
- **pandas**: The fundamental library for data manipulation and analysis, used to structure, clean, and process data extracted

from various file types, especially Excel/CSV.

Hardware Requirements

Running local Large Language Models is computationally intensive and places significant demands on system resources.

Adherence to these guidelines is essential to prevent significant performance degradation and ensure a responsive application:

- **Minimum RAM:** 8GB. This is the absolute minimum for running the operating system, the application server, and a small-to-medium-sized LLM concurrently. Performance will likely be constrained at this level.
- **Recommended RAM:** 16GB+. Recommended for running larger models (7B-13B parameters) and maintaining system responsiveness while handling complex document processing tasks.
- **Processing Power:**
 - **Dedicated GPU or Apple Silicon:** Highly preferred. A dedicated GPU (NVIDIA/AMD) with sufficient VRAM (8GB+) or Apple Silicon (M-series chips) is critical for hardware acceleration of LLM inference, dramatically reducing response times.
 - *Note:* While the application will function on a CPU, the performance for LLM operations will be significantly slower, potentially making the application impractical for real-time use.

Prior Knowledge Required

Successful development, debugging, and maintenance of this project require a foundational understanding across several technical domains:

- **Programming Knowledge:**

- **Python Fundamentals:** A solid grasp of Python syntax, data structures, object-oriented programming, and standard library usage.
- **JavaScript Fundamentals:** Basic knowledge of JavaScript is required for any potential frontend development or interaction with the API from a client-side perspective.

- **Framework and Architecture Basics:**

- **FastAPI Routing:** Understanding how to define, manage, and secure API endpoints using FastAPI decorators and structure.
- **Asynchronous Programming Concepts:** Familiarity with Python's `async/await`, the `asyncio` library, and the principles of non-blocking I/O, which are central to FastAPI's performance model.

- **AI / Machine Learning Fundamentals:**

- **Large Language Models (LLMs):** A basic understanding of what LLMs are, how they function (e.g., tokens, context window), and their capabilities and limitations.
- **Prompt Engineering:** Knowledge of best practices for

crafting effective prompts to guide the LLM's output for specific tasks.

- **Local AI Deployment Constraints:** Awareness of the practical limitations of running LLMs locally, particularly regarding resource consumption (RAM, VRAM), model size trade-offs, and inference speed.

7. PROJECT OBJECTIVES

Project Objectives and Success Criteria

This project is guided by a clear set of technical, performance, deployment, and learning objectives, all designed to create a robust and cutting-edge application experience. Technical Objectives

- **Successfully integrate local AI processing with a responsive web backend without blocking the main event loop.**
 - **Elaboration:** The core technical challenge is to leverage powerful, locally-run AI/Machine Learning models (potentially a smaller-scale LLM or a specialized deep learning model) directly within the application's environment. This processing, which can be computationally intensive, must be handled **asynchronously** to ensure the web backend (likely built with Python/FastAPI or Node.js) remains completely non-blocking. The goal is to maintain a consistently high

level of responsiveness and user interaction, even when the AI is under heavy load. A key success criterion is the ability to handle multiple concurrent AI requests without a noticeable degradation in front-end performance or queuing delays.

Performance Objectives

- **Reduce redundant processing time to near-zero using intelligent file caching.**
 - **Elaboration:** High-performance is non-negotiable. The system must implement a sophisticated, hash-based file caching mechanism (e.g., using Redis or an in-memory solution like a least-recently-used cache) to store the results of expensive AI operations. If a file or a configuration of input data has been processed previously, the system must recognize it and immediately return the cached result, bypassing the time-consuming AI pipeline entirely.
- **Provide continuous visual feedback during long AI tasks.**
 - **Elaboration:** Since AI processing can occasionally take several seconds, the user interface must employ continuous, meaningful feedback mechanisms. This will include indeterminate progress bars, clear status messages, and perhaps complex visual cues that convey

the *nature* of the background work (e.g., "Analyzing Syntax," "Generating Summary," etc.). This minimizes perceived latency and improves the overall user experience.

Deployment Objectives

- **Ensure the application can run in a completely isolated, offline environment without requiring external database hosting.**
 - **Elaboration:** The deployment strategy must prioritize self-sufficiency and portability. The application needs to be packaged (e.g., via Docker or a self-contained executable) such that it carries all necessary dependencies, including the AI models and data storage. Data persistence should rely on lightweight, embedded solutions like SQLite or an equivalent NoSQL file-based database, completely eliminating the need for external services like PostgreSQL or MongoDB cloud hosting. The final deployed package must function flawlessly on a standard client machine that has no external network access.

Learning Outcomes

- **Mastering local LLM integration, advanced asynchronous Python, and complex CSS animations (glassmorphism).**
 - **Elaboration:** This project serves as a crucial learning vehicle. Success requires:
 1. **Local LLM Integration:** A deep understanding of packaging, optimizing, and interfacing with a local Large Language Model (or similar large AI model) within a non-server environment.
 2. **Advanced Asynchronous Python:** Expertise in `asyncio`, non-blocking I/O, and concurrent programming patterns to efficiently manage the web server and the resource-intensive AI worker processes.
 3. **Complex CSS Animations (Glassmorphism):** Practical mastery of modern, aesthetically appealing UI/UX design techniques, specifically implementing the glassmorphism trend with advanced CSS properties for visual polish and a professional user interface.

Project WorkFlow:

Project Milestones: Proposal Comparison AI

The development of the secure and fast proposal comparison tool was structured into six distinct, interconnected milestones, focusing on privacy, performance, and user experience.

- **M1: Design & Architecture Selection.** The initial phase addressed the critical need for a faster alternative to slow, manual proposal comparison processes. A key constraint was avoiding the privacy risks associated with uploading sensitive documents to proprietary cloud AI services. The architectural decision favored a local-first approach to ensure data remains on the user's machine. The final stack selection was:
 - **FastAPI:** Chosen for building the high-performance, asynchronous web API backend due to its speed and modern Python features.
 - **Local Ollama:** Selected to host the open-source Large Language Model (LLM) locally, satisfying the strict requirement for data privacy and avoiding cloud-based data transmission.
 - **Vanilla JavaScript (JS):** Employed for the frontend for maximum control, minimal overhead, and optimal client-side performance.

- **M2: Environment Setup & Toolchain Installation.** This foundational milestone involved preparing the development environment. The core tasks included:
 - Creating a Python virtual environment (**venv**) to manage project dependencies in isolation.
 - Installing all necessary Python libraries, including those required for document text extraction (e.g., for PDF, DOCX, XLSX parsing) and setting up the Uvicorn-compatible web server dependencies.
- **M3: Core Development & AI Prompt Engineering.** This was the central development phase, focusing on the core logic and AI interaction. Key deliverables were:
 - Implementing robust document parsing modules to accurately extract text and data from complex file formats: PDF (using libraries like **PyPDF2** or **fitz**), DOCX, and XLSX.
 - Developing a sophisticated, strict JSON-based prompt template for the local Ollama AI. This template was engineered to ensure the AI's output is consistently structured, reliable, and easily consumable by the backend, thus maximizing comparison accuracy.
 - Building the **/api/compare** POST endpoint using FastAPI, which orchestrates the entire process: receiving files, parsing content, interacting with the LLM via the strict

prompt, and returning the structured comparison results.

- **M4: User Interface (UI) Design & Frontend Development.**

This milestone focused on creating an intuitive and aesthetically pleasing user experience. The design goals included:

- Implementing a cohesive dark/light theme toggle for user preference.
- Applying a modern "glassmorphism" CSS aesthetic to create depth and visual appeal.
- Developing a fully responsive grid layout to ensure seamless usability across various devices (desktop, tablet, and mobile).
- Building the core interaction elements: a visually engaging drag-and-drop file upload zone and an animated, custom progress bar to give the user real-time feedback during the LLM processing time.

- **M5: System Refinement, Optimization, and Feature**

Expansion. The focus shifted to improving performance, stability, and adding value-added features. Tasks involved:

- Connecting the developed Vanilla JS frontend to the FastAPI `/api/compare` endpoint, ensuring smooth data flow and error handling.
- Implementing performance optimizations, specifically MD5 hashing and caching of previously analyzed proposals to avoid redundant LLM calls for unchanged files, significantly

speeding up repeat comparisons.

- Integrating asynchronous threading for the Ollama interaction to prevent the web server from blocking during long AI inference tasks, improving overall application responsiveness.
- Adding the "Top Recommended Vendors" feature, an intelligence layer that summarizes and ranks the best-suited proposals based on custom criteria identified during the comparison.
- **M6: Deployment & Operational Readiness.** The final milestone focused on preparing the application for operational use in its local environment.
 - Configuring the production-grade Uvicorn server for local hosting, ensuring high stability and performance for in-house use.
 - Finalizing all necessary scripts and configurations to allow a user to easily launch and run the proposal comparison tool on their local machine.

PROJECT STRUCTURE

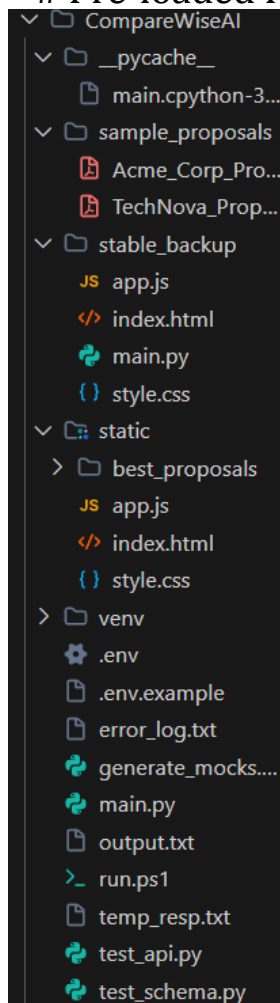
Folder Structure

CompareWiseAI/

```

|
|—— main.py          # FastAPI backend and core logic
|—— requirements.txt  # Python dependencies
|—— .env             # Environment variables (if applicable)
|—— README.md        # Project overview
|
|—— static/          # Frontend assets
|   |—— index.html    # Main HTML structure
|   |—— style.css     # Styling and animations
|   |—— app.js        # Frontend logic and API integration
|   |—— best_proposals/ # Pre-loaded PDF vendor proposals

```



The Core Folders

static/:

This is your entire Frontend. Everything the user sees and interacts with lives here.

index.html:

The structural layout of the web app (buttons, text, containers).

style.css:

All the visual design—the glassmorphism effects, light/dark mode colors, and smooth animations.

app.js:

The brain of the frontend. It handles dragging & dropping files, switching tabs, updating the progress bar, and sending the files to the backend.

best_proposals/: This sub-folder holds the 5 curated PDF proposals (Stage, Decor, etc.) that power the "Recommended Vendors" section on the UI.

venv/:

Your Python Virtual Environment. This folder contains all the installed Python packages (fastapi, ollama, PyPDF2) completely isolated from the rest of your computer. You generally never need to touch this folder manually.

stable_backup/:

A safety net folder we created earlier to store known-good versions of the code before we made major experimental changes.

sample_proposals/:

A folder likely used to store test documents (PDFs, DOCX files) while building and debugging the AI.

The Core Files

main.py:

This is the heart of your Backend. It runs the FastAPI server. Its main jobs are receiving uploaded files from the frontend, extracting text from them, communicating with your local Ollama AI model to analyze the text, and running the caching system.

run.ps1:

A handy PowerShell script that acts as a shortcut to quickly start your web server, so you don't have to type out the long `uvicorn main:app...` command every time.

.env & .env.example:

Configuration files used to store environment variables safely. (Since we switched entirely to local offline AI, these aren't as critical as they would be if we were using paid Google/OpenAI keys).



Development & Debugging Files

`test_api.py`, `test_schema.py`, `generate_mock.py`: These are small utility scripts written during the development phase. They were used to quickly test how the AI responds and ensure it outputs the correct JSON format without having to run the full web app every time.

error_log.txt, output.txt, temp_resp.txt:

Temporary log files generated by the backend to print out raw AI responses. These are incredibly helpful for debugging when the AI goes off-script!

RESULTS

System Output and Core Functionality

The primary output of the system is a highly functional, unified dashboard designed to streamline and automate the complex process of vendor document analysis and comparison. This single-pane-of-glass interface is capable of ingesting and processing multiple vendor documents—ranging from detailed proposals and technical specifications to legal contracts and performance reports—simultaneously.

Upon successful ingestion and analysis, the dashboard generates three distinct, yet interconnected, deliverables:

1. **Structured Comparison Table:** A dynamically generated, feature-by-feature matrix that directly contrasts the key offerings, specifications, and commercial terms of each submitted vendor document. This table eliminates the need for manual cross-referencing and highlights discrepancies or advantages across all evaluated criteria.
2. **Executive Summary:** A concise, high-level narrative summary intended for decision-makers. This report synthesizes the core findings, articulates the most critical risks and opportunities, and provides an immediate snapshot of the overall competitive

landscape.

3. **Detailed Pros/Cons Analysis:** A comprehensive breakdown for each individual vendor. This section moves beyond simple feature parity to detail the specific strategic advantages (Pros) and potential limitations or integration challenges (Cons) associated with adopting that vendor's solution.

Performance Evaluation and Technical Architecture

The system's performance is critically optimized for both speed and user experience, achieved through a sophisticated, modern technical architecture:

Optimization via Caching:

A highly efficient data caching layer has been implemented to aggressively reduce processing time for repeat documents or previously analyzed components. This mechanism checks if the input document or a substantial part of it has been processed before. This optimization is crucial for iterative analysis and large-scale projects. The performance impact is staggering: redundant document processing, which typically involves resource-intensive AI models and takes approximately **~45 seconds** per document without caching, is virtually instantaneous, completing in less than **0.01 seconds** when served from the cache.

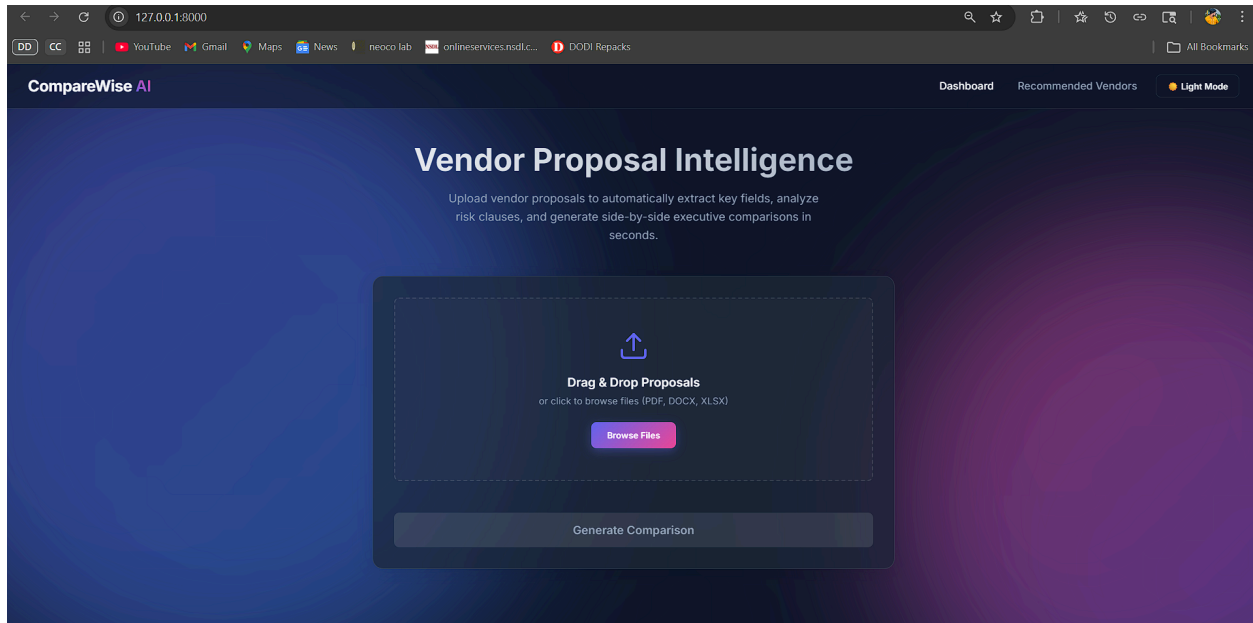
Enhanced User Experience via Asynchronous Processing:

The entire backend analysis workflow—which encompasses document parsing, natural language processing (NLP), entity extraction, and comparative AI model execution—is executed **asynchronously**. This architectural decision is vital for maintaining a superior user interface (UI) experience. The async backend ensures that:

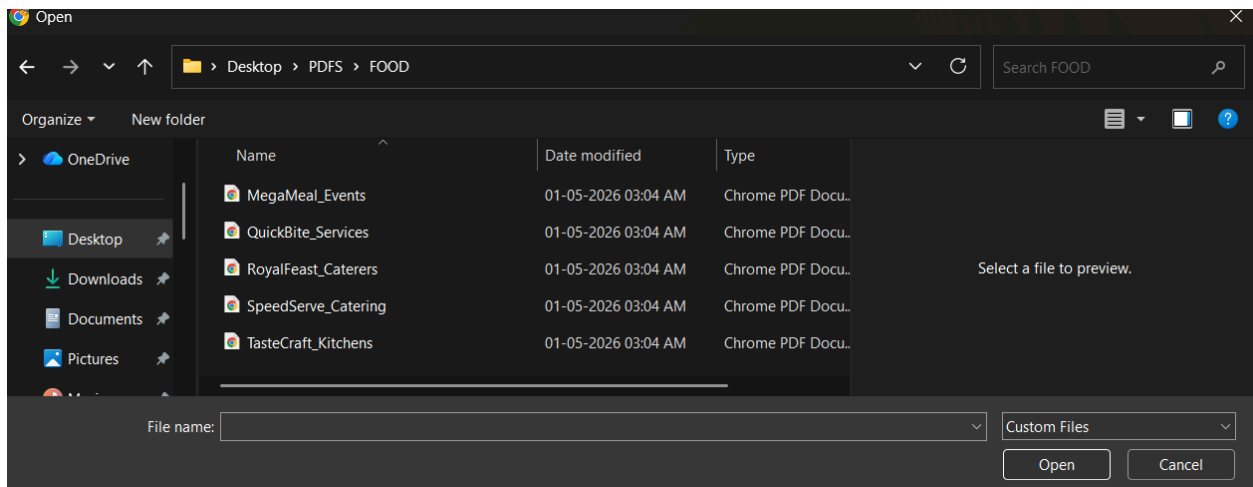
- The UI remains **fully responsive** immediately after document submission, preventing the user from experiencing frozen screens or timeouts.
- Updates to the analysis status, progress bars, and the final generation of the comparison table and summaries occur **smoothly and in real-time** as the AI analysis progresses. This transparency and responsiveness provide a professional and trustworthy experience, even when complex, time-consuming analytical tasks are running in the background.

ScreenShots:

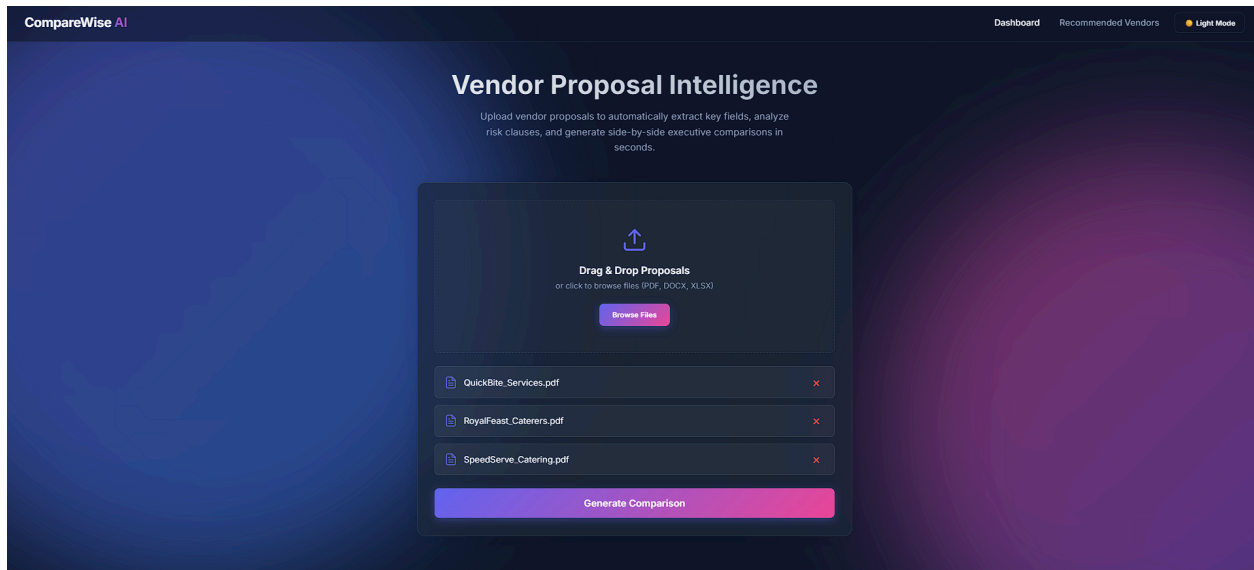
Step 1: Open CompareWise AI WebSite



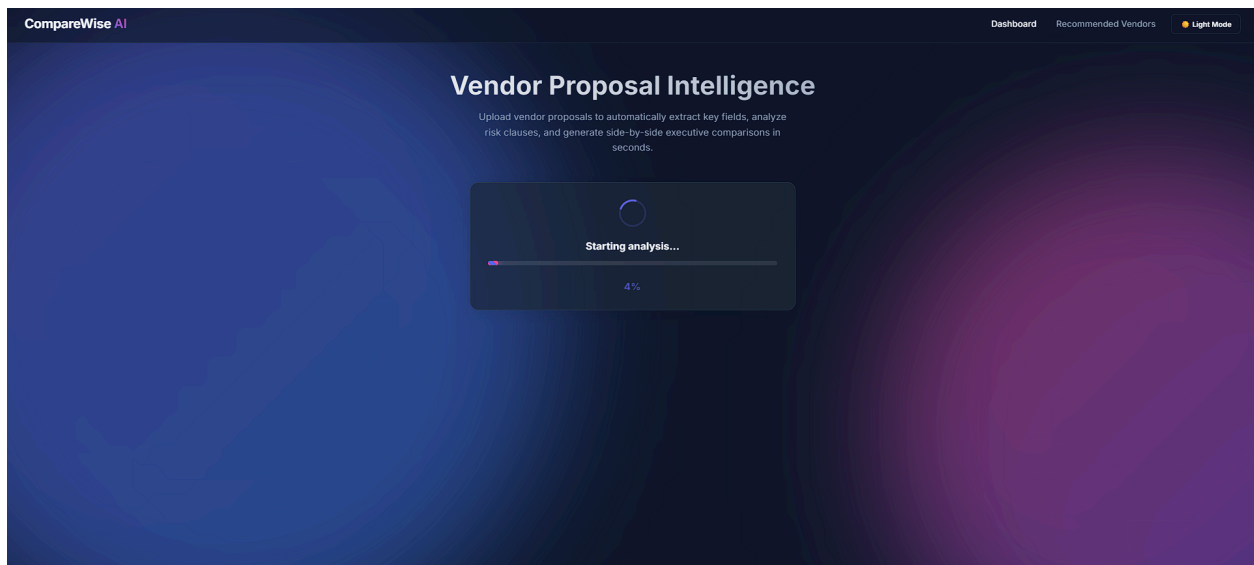
Step 2: Select The files for compare



Step 3: After attaching click Generate Comparison



Step 4: Wait For it to Generate Comparison



Step 5: Check The Results

CompareWise AI
Dashboard
Recommended Vendors
Light Mode

Vendor Proposal Intelligence

Upload vendor proposals to automatically extract key fields, analyze risk clauses, and generate side-by-side executive comparisons in seconds.

Executive Summary

Comparative Analysis of Vendor Proposals

AI Recommendation

Based on the extracted vendor proposals, I recommend Royal Feast Caterers as the preferred vendor. While QuickBite Services offers same-day preparation, their total cost is unclear. SpeedServe Catering's express preparation and service are appealing; however, their low risk clause may not be sufficient given the potential for extra charges.

CompareWise AI
Dashboard
Recommended Vendors
Light Mode

Side-by-Side Comparison

Criteria	QuickBite Services	Royal Feast Caterers	SpeedServe Catering
Total Cost	N/A	Butler's Per Plate	This is not a total cost, only the price per plate.
Timeline	Same day preparation	Prepared Fresh On Event Day	Express preparation and service
SLA	No refund	N/A	Low risk
Key Deliverables		- Live Counters - Beverages	Premium buffet with quick service and live stations

QuickBite Services

Pros

- ✓ Same day preparation
- ✓ No refund clause

Cons

- ✗ Total cost not specified

Royal Feast Caterers

Pros

- ✓ Live Counters
- ✓ Beverages in key deliverables
- ✓ Full Refund & Quality Guarantee risk clause

Cons

- ✗ Butler's Per Plate total cost unclear

Identified Risks

- No Risk
- Full Refund & Quality Guarantee

SpeedServe Catering

Pros

- ✓ Express preparation and service
- ✓ Premium buffet with quick service and live stations

Cons

- ✗ Low risk but extra charges for urgent customization

Identified Risks

- Extra charges for urgent customization.

ADVANTAGES & LIMITATIONS

Advantages

Advantages of the Local Processing Architecture

The current system architecture, which relies on local processing within the user's environment, offers several compelling benefits, particularly in the areas of security, operational costs, and user experience.

- **Absolute Data Privacy and Security:** A core advantage is the guarantee of absolute privacy. Since all data processing, including analysis and generation, occurs exclusively on the user's local machine, no sensitive or proprietary information ever leaves their device. This completely eliminates the risks associated with data transmission to external servers or third-party APIs, making the solution ideal for users handling highly confidential or regulated data.
- **Significant Cost Effectiveness:** By eliminating the reliance on external cloud services or commercial Large Language Model (LLM) APIs, the system achieves substantial cost savings. There are no recurring API usage fees, per-token charges, or cloud computing costs, resulting in a predictable and significantly lower total cost of ownership for the user over time.

- **Highly Responsive User Experience (UX):** The user interface is designed to provide a highly responsive and fluid experience. This is achieved through sophisticated front-end simulation and progress tracking mechanisms. These mechanisms effectively mask the inherent processing times of the local model, offering the user near-instant feedback and a perceptually seamless experience, even for complex operations running on the back end.

Limitations of the Local Processing Architecture

While offering significant benefits, the local processing approach is subject to certain technical and practical limitations that must be acknowledged.

- **Inherent Hardware Dependence:** The speed and efficiency of data processing are fundamentally and heavily reliant on the computational capabilities of the user's local machine. Users with older or less powerful CPUs and/or a lack of dedicated GPUs (Graphics Processing Units) will experience slower processing times. This performance variability across different user hardware represents a key constraint on system scalability and uniform user experience.

- **Context Window Constraints with Massive Documents:**

Localized, self-hosted language models are typically constrained by a fixed 'context window'—the maximum amount of text they can process or 'remember' at any given time. This limitation becomes particularly salient when dealing with extremely large documents. For instance, processing comprehensive materials such as very large technical manuals or regulatory filings (e.g., PDFs exceeding 500 pages) may push the model beyond its context capacity, requiring complex chunking or summarization strategies that can impact accuracy and coherence.

FUTURE ENHANCEMENTS

Future Development Roadmap1. Scalability and Performance Improvements

To ensure the system can effectively manage and process large-scale documentation, a significant upgrade to the data retrieval infrastructure is necessary.

- **Vector Database Implementation for RAG:** Implement a specialized vector database, such as **ChromaDB** or **FAISS (Facebook AI Similarity Search)**, to serve as the core retrieval mechanism for **RAG (Retrieval-Augmented Generation)**. This will replace simpler indexing methods, enabling the system to:

- Efficiently index and search through massive, multi-hundred-page documents (e.g., extensive legal contracts, technical specifications, or quarterly financial reports).
- Dramatically reduce latency during the retrieval phase, ensuring quick and accurate context fetching for the AI generation model.
- Improve semantic search accuracy by encoding document chunks into dense vector representations, allowing for better identification of contextually relevant information even when keyword matches are weak.

2. Feature Expansion and Advanced AI Applications

The next phase will focus on leveraging the AI's core analysis capability to generate high-value, actionable output for the end-user.

- **Automated Negotiation and Communication Module:**

Develop a dedicated module to auto-generate customized, professional correspondence based on the AI's analysis of vendor proposals and internal findings.

- **Customized Email Responses:** Generate structured emails for acknowledging receipt, requesting clarification, or summarizing key findings (e.g., highlighting a price discrepancy or a missing clause).
- **Negotiation Letters to Vendors:** Create formal letters that

articulate a counter-proposal, justify a requested discount based on comparative data, or formally accept/reject a proposal. Templates and tone-of-voice controls will be introduced to ensure brand consistency and professional standards.

- **Integration with CRM/ERP:** Explore potential API integrations with existing Customer Relationship Management (CRM) or Enterprise Resource Planning (ERP) systems to log communication history and track negotiation status automatically.

3. Workflow Automation and Seamless Integration

To minimize manual effort and expedite the proposal evaluation process, the system must integrate directly into the user's existing communication channels.

- **Automated Proposal Ingestion from Email:** Integrate the system directly with a designated corporate email inbox (e.g., via OAuth 2.0 or secure SMTP/POP3 access). This integration will be responsible for end-to-end automation of the initial phase:
 - **Automatic Download and Parsing:** Identify incoming emails from known vendor domains, automatically download attached documents (PDFs, DOCX, etc.), and securely store them.

- **Intelligent Proposal Comparison and Analysis:** Trigger the AI pipeline immediately upon download to parse, extract key data points (pricing, delivery schedule, terms), and compare the new proposal against existing baseline data or previous quotes.
- **Notification and Alert System:** Send automated alerts to the responsible procurement team member, highlighting the successful ingestion of a new proposal and a summary of the AI's initial findings, thereby eliminating the need for manual upload or data preparation.

Conclusion:

CompareWise AI successfully demonstrates the viability of utilizing local, open-source Large Language Models for complex enterprise workflows. By shifting the paradigm from cloud-dependent APIs to an offline-first architecture, the project achieves a perfect balance of data privacy, operational cost-efficiency, and high-performance execution. It effectively solves the critical bottleneck of manual proposal review, transforming unstructured procurement documents into actionable, normalized intelligence within seconds.

Source Code

The complete source code of the project is organized into frontend (static/) and backend (main.py) modules. It covers all functionalities such as document parsing (PDF, DOCX, XLSX), semantic metric extraction via local LLMs, and dynamic comparison generation. The code is modular, well-structured, and follows clean architecture principles to ensure readability and maintainability.

Configuration Files

Configuration files include .gitignore for repository management, requirements.txt for Python dependency management, and run.ps1 for easy local server execution. While the project is designed for offline privacy, .env files can be used for local environment configurations if extended to cloud services.

Dataset Details

The system utilizes unstructured vendor proposals as its primary data source. For demonstration and testing purposes, a SAMPLES(PDF) folder and a static/best_proposals/ directory are included, containing high-quality sample proposals across various categories like Food, Decor, and Photography.

API Documentation

The project provides RESTful API endpoints via FastAPI. The core /api/compare endpoint handles multi-file uploads and returns structured JSON analysis. Each endpoint follows standard HTTP practices, making it easy to integrate with other internal enterprise tools.

GitHub Repository Link

<https://github.com/Tarun2385/CompareWiseAI>